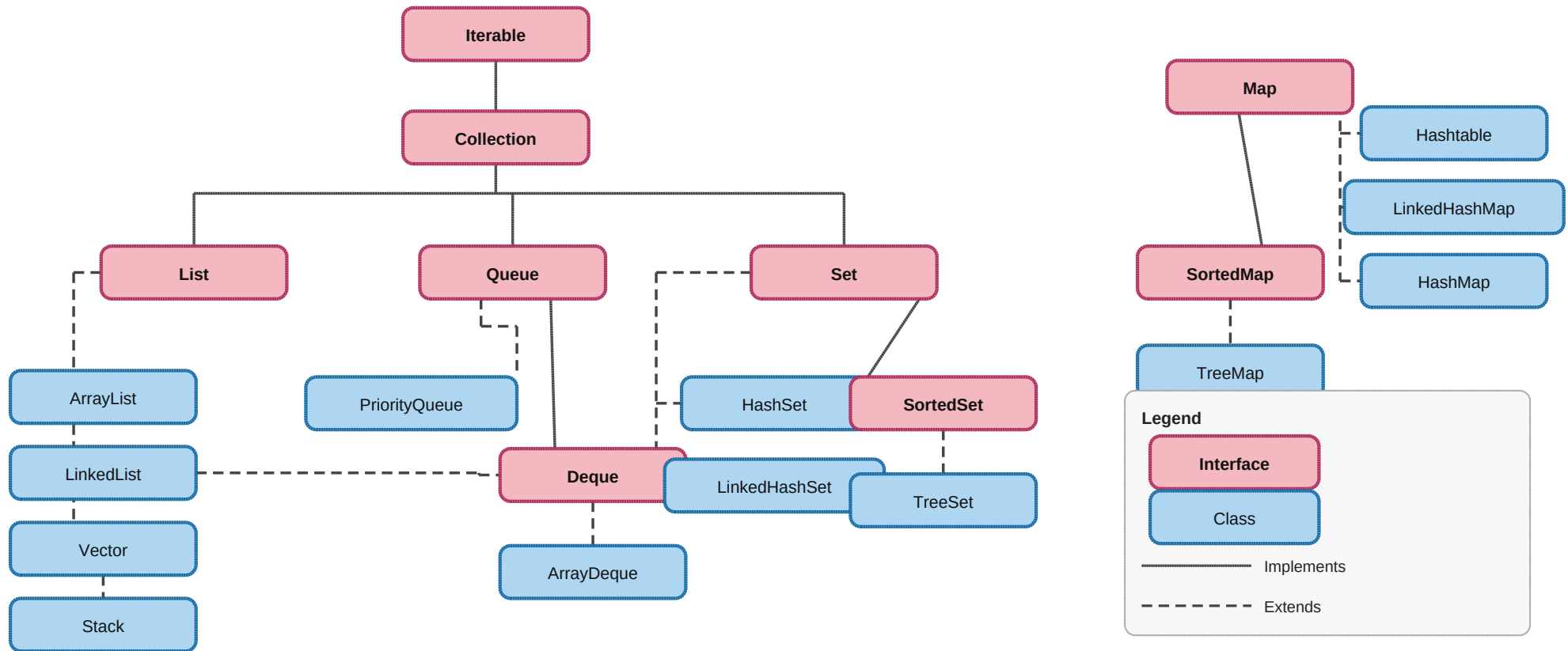


Java Collection Framework -- Visual Hierarchy (Interfaces & Classes)

Java Backend Architecture Interview Series - Chapter 1: Java Collections Deep Dive



Java Collection Framework -- Complete Hierarchy: Short Description & Practical Usage

Java Backend Architecture Interview Series - Chapter 1: Java Collections Deep Dive

Interface	Abstract Class	Concrete Class	Concurrent/Thread-Safe	Deprecated/Legacy
Class / Interface (indent = hierarchy depth)	Short Description		Practical Usage	
Iterable<T>	Root interface; anything that can be iterated via for-each		Implement to make a custom class usable in enhanced for-loops	
Collection<E>	Core group-of-elements contract: add, remove, contains, size, iterate		Use as the widest parameter/return type when any collection is acceptable	
List<E>	Ordered sequence allowing duplicates and index-based access		Use when insertion order or positional access matters (e.g. pagination)	
ArrayList<E>	Resizable array; O(1) random access, O(n) insert/delete in middle		Default List choice; prefer when reads and index access dominate	
LinkedList<E>	Doubly-linked list; O(1) head/tail ops; also implements Deque		Use as a queue or deque; avoid when random access by index is needed	
Vector<E>	Synchronized resizable array; legacy predecessor of ArrayList		Avoid: use ArrayList + external sync or CopyOnWriteArrayList instead	
Stack<E>	Synchronized LIFO stack extending Vector		Avoid: use ArrayDeque as a stack; it is faster and unsynchronized	
CopyOnWriteArrayList<E>	Thread-safe list; writes copy the whole array, reads are lock-free		Event-listener lists, observer registries where reads vastly outnumber writes	
Set<E>	Unordered collection with no duplicates		Use when uniqueness must be enforced and order is irrelevant	
HashSet<E>	Hash-table backed Set; O(1) average for add/remove/contains		General-purpose deduplication, membership checks, visited-node tracking	
LinkedHashSet<E>	HashSet maintaining insertion order via doubly-linked list		Deduplication when you also need predictable iteration order	
SortedSet<E>	Set whose elements are kept in ascending natural or comparator order		Use as parameter type when callers need a sorted unique collection	
NavigableSet<E>	SortedSet with floor/ceiling/lower/higher navigation methods		Range queries: all items between X and Y, nearest-neighbour lookups	
TreeSet<E>	Red-Black tree backed NavigableSet; O(log n) for all ops		Sorted leaderboards, scheduling by priority, range-scan queries	
ConcurrentSkipListSet<E>	Lock-free skip-list NavigableSet; O(log n) thread-safe ops		Concurrent sorted sets: shared leaderboards, multi-thread range queries	
EnumSet<E>	Bit-vector backed Set for enum constants; O(1) via bitwise ops		Feature-flag sets, permission bitmasks, day-of-week combinations	
CopyOnWriteArraySet<E>	Thread-safe Set; backed by CopyOnWriteArrayList; write copies array		Small read-heavy concurrent sets (e.g. active-session tracking)	
Queue<E>	FIFO collection: offer at tail, poll from head		Use as the narrowest type when only FIFO semantics are needed	
PriorityQueue<E>	Min-heap queue; elements dequeued in priority order, not FIFO		Dijkstra/A* shortest path, task schedulers, top-K element problems	
Deque<E>	Double-ended queue: insert/remove at both head and tail		Use as both stack and queue; undo/redo buffers, sliding-window algorithms	
ArrayDeque<E>	Circular-array backed Deque; O(1) amortized at both ends; no nulls		Preferred stack and queue replacement; BFS/DFS, monotonic-deque problems	
BlockingQueue<E>	Thread-safe queue with blocking put/take for producer-consumer patterns		Thread pool work queues, pipeline stages, rate-limited ingestion	
ArrayBlockingQueue<E>	Bounded blocking FIFO backed by circular array; single lock		Fixed-capacity work queues where bounded memory is required	
LinkedBlockingQueue<E>	Optionally bounded blocking FIFO; separate put/take locks for throughput		ThreadPoolExecutor default queue; high-throughput producer-consumer pipes	
PriorityBlockingQueue<E>	Unbounded blocking min-heap; put never blocks; take blocks if empty		Concurrent task schedulers where priority ordering is needed	

Class / Interface (indent = hierarchy depth)	Short Description	Practical Usage
SynchronousQueue<E>	Zero-capacity handoff channel; each put waits for a take and vice versa	Direct thread-to-thread handoff in <code>Executors.newCachedThreadPool()</code>
DelayQueue<E>	Unbounded queue where elements become available after a delay expires	Scheduled task execution, cache-entry expiry, retry-after timers
LinkedTransferQueue<E>	Unbounded lock-free queue with optional synchronous transfer()	High-performance pipelines where producers optionally wait for consumers
Map<K,V>	Key-value mapping contract: put, get, remove, keySet, entrySet	Use as the widest parameter/return type for any key-value structure
HashMap<K,V>	Hash-table Map; O(1) avg get/put; buckets treeify at 8 collisions	General-purpose key-value store; caches, lookup tables, frequency maps
LinkedHashMap<K,V>	HashMap maintaining insertion or access order; override <code>removeEldestEntry</code> for LRU	LRU caches, audit logs, ordered JSON serialisation
SortedMap<K,V>	Map whose keys are kept in ascending order	Use as parameter type when callers need keys in sorted order
NavigableMap<K,V>	SortedMap with floor/ceiling/lower/higher key navigation	Range-scan indexes, time-series lookups, interval trees
TreeMap<K,V>	Red-Black tree Map; O(log n) all ops; keys always sorted	Range queries on keys, sorted config maps, interval scheduling
ConcurrentSkipListMap<K,V>	Lock-free skip-list NavigableMap; O(log n) concurrent ops	Concurrent time-series stores, shared leaderboards with range access
EnumMap<K,V>	Array-backed Map indexed by enum ordinal; O(1) and very compact	State-machine transitions, strategy-pattern dispatch, flag-to-action maps
WeakHashMap<K,V>	Keys held as <code>WeakReferences</code> ; entries auto-removed when key is GC'd	Metadata caches tied to object lifetime (e.g. class-to-schema maps)
Hashtable<K,V>	Synchronized hash table; legacy; no null keys or values	Avoid: use <code>ConcurrentHashMap</code> for thread safety or <code>HashMap</code> otherwise
Properties	<code>Hashtable<String,String></code> with load/store for .properties files and XML	App configuration files, i18n message bundles, system property overrides
ConcurrentMap<K,V>	Thread-safe Map contract adding atomic <code>putIfAbsent</code> / <code>replace</code> / <code>remove</code>	Use as parameter type for any concurrent map to stay implementation-agnostic
ConcurrentHashMap<K,V>	CAS reads + bin-level synchronized writes; high-concurrency O(1) avg	Shared caches, frequency counters, connection pools, session registries

Diagram 1 – Key Methods, Descriptions & Time Complexities

Java Backend Architecture Interview Series - Chapter 1: Java Collections Deep Dive

Interface	Abstract Class	Concrete Class	Concurrent/Thread-Safe	Deprecated/Legacy
Class / Interface	Key Methods Short Description Time Complexity			
Iterable<T>	iterator() – Returns an Iterator over elements – O(1) forEach(Consumer) – Performs action for each element – O(n) spliterator() – Creates a Spliterator for parallel ops – O(1)			
Collection<E>	add(E e) – Adds element; returns true if changed – O(varies) remove(Object o) – Removes first occurrence of element – O(varies) contains(Object o) – Returns true if element exists – O(varies) size() – Returns number of elements – O(1) isEmpty() – Returns true if no elements – O(1) clear() – Removes all elements – O(n) iterator() – Returns Iterator over elements – O(1) addAll(Collection c) – Adds all elements from collection – O(k) removeAll(Collection c) – Removes all elements in collection – O(n*k) retainAll(Collection c) – Keeps only elements also in collection – O(n*k) toArray() – Returns Object[] of all elements – O(n) containsAll(Collection c) – Returns true if all elements present – O(n*k)			
List<E>	get(int index) – Returns element at position – O(varies) set(int index, E e) – Replaces element at position; returns old – O(varies) add(int index, E e) – Inserts element at position – O(varies) remove(int index) – Removes element at position; returns it – O(varies) indexOf(Object o) – Returns first index of element; -1 if absent – O(n) lastIndexOf(Object o) – Returns last index of element; -1 if absent – O(n) subList(int from, int to) – Returns view of portion [from,to) – O(1) listIterator() – Returns a ListIterator (bidirectional) – O(1) listIterator(int index) – Returns ListIterator starting at index – O(1) sort(Comparator c) – Sorts list using comparator (TimSort) – O(n log n) replaceAll(UnaryOperator) – Replaces each element with operator result – O(n)			
ArrayList<E>	get(int index) – Returns element at index – O(1) set(int index, E e) – Replaces element at index – O(1) add(E e) – Appends to end; resizes if needed (amortized) – O(1) amortized add(int index, E e) – Inserts at index; shifts right elements – O(n) remove(int index) – Removes at index; shifts left elements – O(n) remove(Object o) – Removes first occurrence by equality – O(n) contains(Object o) – Linear scan for element – O(n) indexOf(Object o) – Linear scan; returns first index – O(n) size() – Returns element count – O(1) ensureCapacity(int min) – Pre-allocates capacity to avoid repeated resizing – O(n) trimToSize() – Shrinks backing array to current size – O(n) subList(int from, int to) – Returns a view (backed by this list) – O(1) clear() – Removes all elements; nullifies refs – O(n) toArray() – Copies elements into new Object[] – O(n)			

Class / Interface	Key Methods Short Description Time Complexity
LinkedList<E>	add(E e) / addLast(E e) – Appends node to tail – $O(1)$ addFirst(E e) – Prepends node to head – $O(1)$ get(int index) – Traverses from head or tail to index – $O(n)$ getFirst() – Returns head element without removal – $O(1)$ getLast() – Returns tail element without removal – $O(1)$ remove(int index) – Unlinks node at index – $O(n)$ removeFirst() – Unlinks and returns head node – $O(1)$ removeLast() – Unlinks and returns tail node – $O(1)$ offer(E e) – Enqueues at tail (Queue method) – $O(1)$ poll() – Dequeues from head; null if empty – $O(1)$ peek() – Returns head without removal; null if empty – $O(1)$ push(E e) – Pushes onto stack (= addFirst) – $O(1)$ pop() – Pops from stack (= removeFirst) – $O(1)$ contains(Object o) – Linear scan for element – $O(n)$
Vector<E>	get(int index) – Returns element at index (synchronized) – $O(1)$ set(int index, E e) – Replaces element (synchronized) – $O(1)$ add(E e) – Appends to end; doubles capacity on resize (sync) – $O(1)$ amortized add(int index, E e) – Inserts at index; shifts right (synchronized) – $O(n)$ remove(int index) – Removes at index (synchronized) – $O(n)$ elementAt(int index) – Legacy get (synchronized) – $O(1)$ insertElementAt(E, int) – Legacy insert (synchronized) – $O(n)$ removeElementAt(int) – Legacy remove (synchronized) – $O(n)$ capacity() – Returns current backing array length – $O(1)$ ensureCapacity(int min) – Ensures capacity $\geq$ min (synchronized) – $O(n)$ trimToSize() – Shrinks to current size (synchronized) – $O(n)$ indexOf(Object o) – Linear scan (synchronized) – $O(n)$ size() – Returns element count (synchronized) – $O(1)$
Stack<E>	push(E item) – Pushes item onto top of stack – $O(1)$ sync pop() – Removes and returns top element – $O(1)$ sync peek() – Returns top without removal – $O(1)$ sync empty() – Returns true if stack has no elements – $O(1)$ sync search(Object o) – 1-based position from top; -1 if absent – $O(n)$ sync get(int index) – Inherited from Vector – $O(1)$ sync add(E e) – Inherited from Vector (appends) – $O(1)$ sync remove(int index) – Inherited from Vector – $O(n)$ sync size() – Inherited from Vector – $O(1)$ sync contains(Object o) – Inherited from Vector – $O(n)$ sync
CopyOnWriteArrayList<E>	get(int index) – Reads from current snapshot array – $O(1)$ set(int index, E e) – Copies array, sets element, publishes – $O(n)$ add(E e) – Copies array, appends, publishes – $O(n)$ add(int index, E e) – Copies array, inserts at index, publishes – $O(n)$ remove(int index) – Copies array without element, publishes – $O(n)$ remove(Object o) – Scans, copies without first match, publishes – $O(n)$ contains(Object o) – Linear scan on snapshot – $O(n)$ indexOf(Object o) – Linear scan; returns first index – $O(n)$ addIfAbsent(E e) – Adds only if not already present – $O(n)$ addAllAbsent(Collection) – Adds elements not already in list – $O(n*k)$ size() – Returns snapshot array length – $O(1)$ iterator() – Snapshot iterator; never throws CME – $O(1)$ clear() – Replaces backing array with empty array – $O(1)$

Class / Interface	Key Methods Short Description Time Complexity
Set<E>	add(E e) – Adds element; false if already present – O(varies) remove(Object o) – Removes element; false if absent – O(varies) contains(Object o) – Returns true if element present – O(varies) size() – Returns number of unique elements – O(1) isEmpty() – Returns true if no elements – O(1) clear() – Removes all elements – O(n) iterator() – Returns Iterator over elements – O(1) addAll(Collection c) – Union: adds all missing elements – O(k) removeAll(Collection c) – Difference: removes matching elements – O(n*k) retainAll(Collection c) – Intersection: keeps only matching elements – O(n*k) toArray() – Returns Object[] snapshot – O(n) containsAll(Collection c) – Returns true if all elements present – O(n*k)
HashSet<E>	add(E e) – Hashes key; adds to bucket; false if duplicate – O(1) avg remove(Object o) – Hashes key; unlinks from bucket – O(1) avg contains(Object o) – Hashes key; scans bucket chain – O(1) avg size() – Returns element count – O(1) isEmpty() – Returns true if count == 0 – O(1) clear() – Clears all buckets; resets size – O(capacity) iterator() – Iterates over all bucket chains – O(1) init addAll(Collection c) – Adds all elements (may trigger resize) – O(k) removeAll(Collection c) – Removes all matching elements – O(n) retainAll(Collection c) – Keeps only matching; removes rest – O(n) toArray() – Returns unsorted Object[] snapshot – O(n) clone() – Shallow copy of HashSet and its map – O(n)
LinkedHashSet<E>	add(E e) – Adds to hash bucket + appends to doubly-linked list – O(1) avg remove(Object o) – Removes from bucket + unlinks from list – O(1) avg contains(Object o) – Hash lookup in bucket – O(1) avg size() – Returns element count – O(1) isEmpty() – Returns true if count == 0 – O(1) clear() – Clears buckets and linked list – O(n) iterator() – Iterates in insertion order – O(1) init addAll(Collection c) – Adds maintaining insertion order – O(k) removeAll(Collection c) – Removes matching; preserves order of rest – O(n) retainAll(Collection c) – Keeps matching in insertion order – O(n) toArray() – Returns insertion-ordered Object[] – O(n) clone() – Shallow copy preserving insertion order – O(n)
SortedSet<E>	first() – Returns smallest element (head) – O(log n) last() – Returns largest element (tail) – O(log n) comparator() – Returns Comparator; null if natural ordering – O(1) headSet(E toElement) – View of elements < toElement – O(log n) tailSet(E fromElement) – View of elements >= fromElement – O(log n) subSet(E from, E to) – View of elements in [from, to) – O(log n) add(E e) – Inserts in sorted position – O(log n) remove(Object o) – Removes element maintaining order – O(log n) contains(Object o) – Binary search / tree lookup – O(log n) iterator() – Iterates in ascending sorted order – O(1) init size() – Returns element count – O(1)

Class / Interface	Key Methods Short Description Time Complexity
NavigableSet<E>	<code>floor(E e)</code> – Greatest element $\leq e$; null if none – $O(\log n)$ <code>ceiling(E e)</code> – Least element $\geq e$; null if none – $O(\log n)$ <code>lower(E e)</code> – Greatest element strictly $< e$ – $O(\log n)$ <code>higher(E e)</code> – Least element strictly $> e$ – $O(\log n)$ <code>pollFirst()</code> – Removes and returns least element – $O(\log n)$ <code>pollLast()</code> – Removes and returns greatest element – $O(\log n)$ <code>descendingSet()</code> – View of set in reverse order – $O(1)$ <code>descendingIterator()</code> – Iterator in descending order – $O(1)$ <code>headSet(E e, boolean incl)</code>– View of elements up to e (inclusive flag) – $O(\log n)$ <code>tailSet(E e, boolean incl)</code>– View of elements from e (inclusive flag) – $O(\log n)$ <code>subSet(E, bool, E, bool)</code> – View with configurable inclusive bounds – $O(\log n)$ <code>first()</code> / <code>last()</code> – Inherited: head / tail element – $O(\log n)$
TreeSet<E>	<code>add(E e)</code> – Inserts via Red-Black tree; $O(\log n)$ rotations if needed – $O(\log n)$ <code>remove(Object o)</code> – Deletes node; rebalances Red-Black tree – $O(\log n)$ <code>contains(Object o)</code> – Binary search down tree – $O(\log n)$ <code>first()</code> – Returns leftmost (minimum) node – $O(\log n)$ <code>last()</code> – Returns rightmost (maximum) node – $O(\log n)$ <code>floor(E e)</code> – Greatest element $\leq e$ – $O(\log n)$ <code>ceiling(E e)</code> – Least element $\geq e$ – $O(\log n)$ <code>lower(E e)</code> – Greatest element strictly $< e$ – $O(\log n)$ <code>higher(E e)</code> – Least element strictly $> e$ – $O(\log n)$ <code>pollFirst()</code> – Removes and returns minimum element – $O(\log n)$ <code>pollLast()</code> – Removes and returns maximum element – $O(\log n)$ <code>headSet(E e)</code> – View of elements $< e$ (backed by tree) – $O(\log n)$ <code>tailSet(E e)</code> – View of elements $\geq e$ (backed by tree) – $O(\log n)$ <code>size()</code> – Returns element count – $O(1)$
ConcurrentSkipListSet<E>	<code>add(E e)</code> – CAS-based insert into skip list – $O(\log n)$ avg <code>remove(Object o)</code> – CAS-based deletion from skip list – $O(\log n)$ avg <code>contains(Object o)</code> – Lock-free search through skip list levels – $O(\log n)$ avg <code>first()</code> – Returns head of skip list – $O(1)$ <code>last()</code> – Traverses to tail of skip list – $O(\log n)$ <code>floor(E e)</code> – Lock-free greatest $\leq e$ search – $O(\log n)$ <code>ceiling(E e)</code> – Lock-free least $\geq e$ search – $O(\log n)$ <code>lower(E e)</code> – Lock-free greatest strictly $< e$ – $O(\log n)$ <code>higher(E e)</code> – Lock-free least strictly $> e$ – $O(\log n)$ <code>pollFirst()</code> – Atomically removes head – $O(\log n)$ <code>pollLast()</code> – Atomically removes tail – $O(\log n)$ <code>size()</code> – Counts traversal (not $O(1)$) – $O(n)$ <code>iterator()</code> – Weakly consistent; no CME – $O(1)$ init
EnumSet<E>	<code>add(E e)</code> – Sets bit at $e.ordinal()$ in <code>long[]</code> – $O(1)$ <code>remove(Object o)</code> – Clears bit at $e.ordinal()$ – $O(1)$ <code>contains(Object o)</code> – Tests bit at $e.ordinal()$ – $O(1)$ <code>addAll(Collection c)</code> – Bitwise OR of two bit-vectors – $O(1)$ <code>removeAll(Collection c)</code> – Bitwise AND-NOT of bit-vectors – $O(1)$ <code>retainAll(Collection c)</code> – Bitwise AND of bit-vectors – $O(1)$ <code>clear()</code> – Sets all bits to zero – $O(1)$ <code>size()</code> – Counts set bits (<code>Long.bitCount</code>) – $O(1)$ <code>iterator()</code> – Iterates set bits in declaration order – $O(n)$ <code>EnumSet.of(E...)</code> – Static factory; creates set of given constants – $O(k)$ <code>EnumSet.allOf(Class)</code> – Static factory; all constants in enum type – $O(n)$ <code>EnumSet.noneOf(Class)</code> – Static factory; empty set for enum type – $O(1)$ <code>EnumSet.range(E from, E to)</code>– Static factory; contiguous range of constants – $O(1)$ <code>clone()</code> – Returns copy of this <code>EnumSet</code> – $O(1)$

Class / Interface	Key Methods Short Description Time Complexity
CopyOnWriteArraySet<E>	add(E e) – Scans array; copies+appends only if absent – $O(n)$ remove(Object o) – Scans; copies array without match – $O(n)$ contains(Object o) – Linear scan of snapshot – $O(n)$ addAll(Collection c) – Adds each element if absent – $O(n*k)$ size() – Returns snapshot array length – $O(1)$ isEmpty() – Returns true if size == 0 – $O(1)$ clear() – Replaces array with empty snapshot – $O(1)$ iterator() – Snapshot iterator; never throws CME – $O(1)$ removeAll(Collection c) – Copies array excluding matching elements – $O(n*k)$ retainAll(Collection c) – Copies array keeping only matching elements – $O(n*k)$ toArray() – Returns copy of snapshot array – $O(n)$
Queue<E>	offer(E e) – Inserts element; returns false if capacity exceeded – $O(\text{varies})$ poll() – Removes and returns head; null if empty – $O(\text{varies})$ peek() – Returns head without removal; null if empty – $O(\text{varies})$ add(E e) – Like offer but throws IllegalStateException if full – $O(\text{varies})$ remove() – Like poll but throws NoSuchElementException if empty – $O(\text{varies})$ element() – Like peek but throws NoSuchElementException if empty – $O(\text{varies})$ size() – Number of elements (inherited from Collection) – $O(\text{varies})$ isEmpty() – True if no elements – $O(\text{varies})$ contains(Object o) – True if element present – $O(\text{varies})$ iterator() – Iterator over elements – $O(1)$
PriorityQueue<E>	offer(E e) / add(E e) – Sifts up from end to maintain heap invariant – $O(\log n)$ poll() / remove() – Removes root; sifts down last element – $O(\log n)$ peek() / element() – Returns root (minimum) without removal – $O(1)$ remove(Object o) – Linear search then sift; removes specific element – $O(n)$ contains(Object o) – Linear scan of backing array – $O(n)$ size() – Returns element count – $O(1)$ comparator() – Returns comparator; null if natural ordering – $O(1)$ toArray() – Returns unsorted copy of backing array – $O(n)$ clear() – Removes all elements; nullifies refs – $O(n)$ iterator() – Arbitrary order iterator (not heap order) – $O(1)$ init addAll(Collection c) – Adds all; may trigger multiple sift-ups – $O(k \log n)$
Deque<E>	offerFirst(E e) – Inserts at head; false if capacity exceeded – $O(1)$ offerLast(E e) – Inserts at tail; false if capacity exceeded – $O(1)$ pollFirst() – Removes and returns head; null if empty – $O(1)$ pollLast() – Removes and returns tail; null if empty – $O(1)$ peekFirst() – Returns head without removal; null if empty – $O(1)$ peekLast() – Returns tail without removal; null if empty – $O(1)$ addFirst(E e) – Like offerFirst but throws if capacity exceeded – $O(1)$ addLast(E e) – Like offerLast but throws if capacity exceeded – $O(1)$ removeFirst() – Like pollFirst but throws if empty – $O(1)$ removeLast() – Like pollLast but throws if empty – $O(1)$ push(E e) – Stack push: equivalent to addFirst – $O(1)$ pop() – Stack pop: equivalent to removeFirst – $O(1)$ descendingIterator() – Iterator from tail to head – $O(1)$

Class / Interface	Key Methods Short Description Time Complexity
ArrayDeque<E>	offerFirst(E e) – Moves head index back; stores element (circular) – O(1) amortized offerLast(E e) – Stores at tail index; advances tail (circular) – O(1) amortized pollFirst() – Returns element at head; advances head index – O(1) pollLast() – Returns element at tail-1; decrements tail index – O(1) peekFirst() – Returns element at head without moving index – O(1) peekLast() – Returns element at tail-1 without moving index – O(1) push(E e) – Stack push = offerFirst – O(1) amortized pop() – Stack pop = pollFirst – O(1) size() – Returns (tail - head) & (array.length - 1) – O(1) contains(Object o) – Linear scan of circular array – O(n) remove(Object o) – Linear scan; shifts elements to fill gap – O(n) clear() – Nullifies all slots; resets head/tail – O(n) toArray() – Copies circular array into linear Object[] – O(n) iterator() – Forward iterator from head to tail – O(1) init
BlockingQueue<E>	put(E e) – Blocks indefinitely if queue full until space available – O(varies) take() – Blocks indefinitely if queue empty until element available – O(varies) offer(E e, long t, TU u) – Waits up to timeout for space; false if timed out – O(varies) poll(long t, TimeUnit u) – Waits up to timeout for element; null if timed out – O(varies) offer(E e) – Non-blocking insert; false if full – O(varies) poll() – Non-blocking remove; null if empty – O(varies) peek() – Non-blocking head view; null if empty – O(varies) remainingCapacity() – Returns space before blocking (MAX_INT if unbounded) – O(1) drainTo(Collection c) – Moves all available elements into collection atomically – O(n) drainTo(Collection c, int) – Moves up to maxElements atomically – O(maxElements) size() – Current element count – O(varies) contains(Object o) – Searches elements (thread-safe) – O(n)
ArrayBlockingQueue<E>	put(E e) – Acquires lock; awaits notFull condition; inserts – O(1) take() – Acquires lock; awaits notEmpty condition; removes – O(1) offer(E e, long t, TU u) – Timed wait on notFull condition – O(1) poll(long t, TimeUnit u) – Timed wait on notEmpty condition – O(1) offer(E e) – Non-blocking; returns false immediately if full – O(1) poll() – Non-blocking; returns null immediately if empty – O(1) peek() – Returns head element without removal – O(1) remainingCapacity() – capacity - size – O(1) drainTo(Collection c) – Moves all elements atomically under lock – O(n) size() – Returns element count – O(1) contains(Object o) – Linear scan under lock – O(n) clear() – Removes all elements under lock – O(n)
LinkedBlockingQueue<E>	put(E e) – Acquires putLock; awaits notFull condition; links node – O(1) take() – Acquires takeLock; awaits notEmpty; unlinks head – O(1) offer(E e, long t, TU u) – Timed wait on putLock notFull condition – O(1) poll(long t, TimeUnit u) – Timed wait on takeLock notEmpty condition – O(1) offer(E e) – Non-blocking; returns false if at capacity – O(1) poll() – Non-blocking; returns null if empty – O(1) peek() – Returns head element without lock – O(1) remainingCapacity() – capacity - count.get() – O(1) drainTo(Collection c) – Acquires both locks; drains all nodes – O(n) size() – Returns count.get() (AtomicInteger) – O(1) contains(Object o) – Acquires both locks; linear scan – O(n) remove(Object o) – Acquires both locks; unlinks node – O(n)

Class / Interface	Key Methods Short Description Time Complexity
PriorityBlockingQueue<E>	<code>put(E e)</code> / <code>offer(E e)</code> – Acquires lock; sifts up in heap (never blocks on put) – $O(\log n)$ <code>take()</code> – Acquires lock; awaits <code>notEmpty</code>; polls heap root – $O(\log n)$ <code>poll(long t, TimeUnit u)</code> – Timed wait for element; sifts down after removal – $O(\log n)$ <code>poll()</code> – Non-blocking; null if empty; sifts down – $O(\log n)$ <code>peek()</code> – Returns heap root without removal – $O(1)$ <code>size()</code> – Returns element count – $O(1)$ <code>remainingCapacity()</code> – Always <code>Integer.MAX_VALUE</code> (unbounded) – $O(1)$ <code>comparator()</code> – Returns comparator; null if natural ordering – $O(1)$ <code>drainTo(Collection c)</code> – Repeatedly polls under lock into collection – $O(n \log n)$ <code>contains(Object o)</code> – Linear scan of heap array under lock – $O(n)$ <code>clear()</code> – Removes all elements under lock – $O(n)$ <code>toArray()</code> – Returns copy of unsorted heap array – $O(n)$
SynchronousQueue<E>	<code>put(E e)</code> – Blocks until another thread calls <code>take()</code> – $O(1)$ <code>take()</code> – Blocks until another thread calls <code>put(E)</code> – $O(1)$ <code>offer(E e, long t, TU u)</code> – Waits up to timeout for a consumer thread – $O(1)$ <code>poll(long t, TimeUnit u)</code> – Waits up to timeout for a producer thread – $O(1)$ <code>offer(E e)</code> – Succeeds only if a thread is already waiting in <code>take()</code> – $O(1)$ <code>poll()</code> – Succeeds only if a thread is already waiting in <code>put()</code> – $O(1)$ <code>peek()</code> – Always returns null (no internal storage) – $O(1)$ <code>isEmpty()</code> – Always returns true – $O(1)$ <code>size()</code> – Always returns 0 – $O(1)$ <code>remainingCapacity()</code> – Always returns 0 – $O(1)$ <code>drainTo(Collection c)</code> – No-op; always transfers 0 elements – $O(1)$ <code>iterator()</code> – Returns empty iterator – $O(1)$
DelayQueue<E>	<code>put(E e)</code> / <code>offer(E e)</code> – Acquires lock; adds to <code>PriorityQueue</code> by delay – $O(\log n)$ <code>take()</code> – Blocks until head element's <code>getDelay()</code> ≤ 0 – $O(\log n)$ <code>poll(long t, TimeUnit u)</code> – Waits up to timeout for an expired element – $O(\log n)$ <code>poll()</code> – Returns head if expired; null otherwise (non-blocking) – $O(\log n)$ <code>peek()</code> – Returns head element regardless of expiry – $O(1)$ <code>size()</code> – Returns <code>PriorityQueue</code> element count – $O(1)$ <code>remainingCapacity()</code> – Always <code>Integer.MAX_VALUE</code> (unbounded) – $O(1)$ <code>drainTo(Collection c)</code> – Transfers all expired elements atomically – $O(n)$ <code>drainTo(Collection c, int)</code> – Transfers up to max expired elements – $O(\max)$ <code>remove(Object o)</code> – Removes specific element under lock – $O(n)$ <code>contains(Object o)</code> – Delegates to <code>PriorityQueue.contains</code> under lock – $O(n)$ <code>clear()</code> – Removes all elements under lock – $O(n)$
LinkedTransferQueue<E>	<code>transfer(E e)</code> – Blocks until a consumer takes the element (sync handoff) – $O(1)$ <code>tryTransfer(E e)</code> – Succeeds only if consumer waiting; else returns false – $O(1)$ <code>tryTransfer(E, long, TU)</code> – Waits up to timeout for a waiting consumer – $O(1)$ <code>put(E e)</code> / <code>offer(E e)</code> – CAS-based enqueue to tail; never blocks – $O(1)$ <code>take()</code> – Blocks until element available; CAS-based dequeue – $O(1)$ <code>poll(long t, TimeUnit u)</code> – Waits up to timeout for head element – $O(1)$ <code>poll()</code> – Non-blocking dequeue; null if empty – $O(1)$ <code>peek()</code> – Returns head without removal – $O(1)$ <code>hasWaitingConsumer()</code> – True if a thread is blocked in <code>take()</code> – $O(1)$ <code>getWaitingConsumerCount()</code> – Approximate count of blocking consumer threads – $O(n)$ <code>size()</code> – Estimated size (traversal; not exact) – $O(n)$ <code>contains(Object o)</code> – Weakly consistent linear scan – $O(n)$

Class / Interface	Key Methods Short Description Time Complexity
Map<K,V>	get(Object key) – Returns value mapped to key; null if absent – O(varies) put(K key, V value) – Associates value with key; returns old value – O(varies) remove(Object key) – Removes mapping; returns old value – O(varies) containsKey(Object key) – Returns true if key has a mapping – O(varies) containsValue(Object val) – Returns true if any key maps to value – O(n) size() – Number of key-value mappings – O(1) isEmpty() – Returns true if no mappings – O(1) clear() – Removes all mappings – O(n) keySet() – Returns Set view of all keys – O(1) values() – Returns Collection view of all values – O(1) entrySet() – Returns Set<Map.Entry> view of all mappings – O(1) getOrDefault(K,V) – Returns value or default if key absent – O(varies) putIfAbsent(K,V) – Puts only if key not already present – O(varies) replace(K,V) – Replaces value only if key is present – O(varies) compute(K,BiFunction) – Computes new value from key+old; removes if null – O(varies) merge(K,V,BiFunction) – Merges value with existing via function – O(varies) forEach(BiConsumer) – Performs action for each entry – O(n) putAll(Map m) – Copies all mappings from m – O(k)
HashMap<K,V>	get(Object key) – Hashes key; walks bucket chain or tree – O(1) avg put(K key, V value) – Hashes key; inserts/updates in bucket; may resize – O(1) avg remove(Object key) – Hashes key; unlinks node from bucket – O(1) avg containsKey(Object key) – Hash lookup in bucket chain – O(1) avg containsValue(Object val) – Full linear scan of all buckets – O(n) size() – Returns entry count field – O(1) isEmpty() – Returns size == 0 – O(1) clear() – Nullifies all bucket arrays; resets size – O(capacity) keySet() – Returns backed Set view of keys – O(1) values() – Returns backed Collection view of values – O(1) entrySet() – Returns backed Set<Entry> view – O(1) getOrDefault(K,V) – Hash lookup; returns default if absent – O(1) avg putIfAbsent(K,V) – Inserts only if key not mapped – O(1) avg compute(K,BiFunction) – Atomically compute and replace value – O(1) avg merge(K,V,BiFunction) – Merge or insert value for key – O(1) avg forEach(BiConsumer) – Iterates all entries and applies action – O(n)
LinkedHashMap<K,V>	get(Object key) – Hash lookup; optionally moves to tail for access order – O(1) avg put(K key, V value) – Inserts in hash bucket + appends to doubly-linked list – O(1) avg remove(Object key) – Removes from bucket + unlinks from doubly-linked list – O(1) avg containsKey(Object key) – Hash lookup – O(1) avg containsValue(Object val) – Iterates doubly-linked list – O(n) size() – Returns entry count – O(1) clear() – Clears buckets and doubly-linked list – O(n) keySet() – Returns keys in insertion/access order – O(1) values() – Returns values in insertion/access order – O(1) entrySet() – Returns entries in insertion/access order – O(1) getOrDefault(K,V) – Hash lookup; default if absent – O(1) avg removeEldestEntry(Entry) – Override to implement LRU eviction policy – O(1) forEach(BiConsumer) – Iterates in insertion/access order – O(n)

Class / Interface	Key Methods Short Description Time Complexity
SortedMap<K,V>	firstKey() – Returns smallest key – $O(\log n)$ lastKey() – Returns largest key – $O(\log n)$ comparator() – Returns Comparator; null if natural ordering – $O(1)$ headMap(K toKey) – View of entries with key < toKey – $O(\log n)$ tailMap(K fromKey) – View of entries with key >= fromKey – $O(\log n)$ subMap(K from, K to) – View of entries in key range [from, to) – $O(\log n)$ get(Object key) – Sorted lookup for key – $O(\log n)$ put(K key, V value) – Inserts in sorted order – $O(\log n)$ remove(Object key) – Removes key maintaining order – $O(\log n)$ keySet() – Returns keys in ascending sorted order – $O(1)$ entrySet() – Returns entries in ascending key order – $O(1)$ values() – Returns values in ascending key order – $O(1)$
NavigableMap<K,V>	floorKey(K key) – Greatest key <= given key; null if none – $O(\log n)$ ceilingKey(K key) – Least key >= given key; null if none – $O(\log n)$ lowerKey(K key) – Greatest key strictly < given key – $O(\log n)$ higherKey(K key) – Least key strictly > given key – $O(\log n)$ floorEntry(K key) – Entry with greatest key <= given key – $O(\log n)$ ceilingEntry(K key) – Entry with least key >= given key – $O(\log n)$ pollFirstEntry() – Removes and returns entry with smallest key – $O(\log n)$ pollLastEntry() – Removes and returns entry with largest key – $O(\log n)$ firstEntry() – Entry with smallest key (no removal) – $O(\log n)$ lastEntry() – Entry with largest key (no removal) – $O(\log n)$ descendingKeySet() – Set view of keys in reverse order – $O(1)$ descendingMap() – Map view in reverse key order – $O(1)$ headMap(K,boolean) – View up to key (inclusive configurable) – $O(\log n)$ tailMap(K,boolean) – View from key (inclusive configurable) – $O(\log n)$
TreeMap<K,V>	get(Object key) – Binary search down Red-Black tree – $O(\log n)$ put(K key, V value) – Inserts node; rebalances Red-Black tree – $O(\log n)$ remove(Object key) – Deletes node; rebalances Red-Black tree – $O(\log n)$ firstKey() / firstEntry() – Returns leftmost (minimum) node – $O(\log n)$ lastKey() / lastEntry() – Returns rightmost (maximum) node – $O(\log n)$ floorKey(K key) – Greatest key <= given key – $O(\log n)$ ceilingKey(K key) – Least key >= given key – $O(\log n)$ lowerKey(K key) – Greatest strictly < given key – $O(\log n)$ higherKey(K key) – Least strictly > given key – $O(\log n)$ pollFirstEntry() – Removes and returns minimum entry – $O(\log n)$ pollLastEntry() – Removes and returns maximum entry – $O(\log n)$ headMap(K toKey) – Backed view of entries with key < toKey – $O(\log n)$ tailMap(K fromKey) – Backed view of entries with key >= fromKey – $O(\log n)$ subMap(K from, K to) – Backed view for key range [from, to) – $O(\log n)$ size() – Returns entry count field – $O(1)$
ConcurrentSkipListMap<K,V>	get(Object key) – Lock-free binary search through skip list – $O(\log n)$ avg put(K key, V value) – CAS-based insert with skip-list promotion – $O(\log n)$ avg remove(Object key) – CAS-based deletion; marks node then unlinks – $O(\log n)$ avg firstKey() – Returns head node key – $O(1)$ lastKey() – Traverses to tail node – $O(\log n)$ floorKey(K key) – Lock-free search for greatest <= key – $O(\log n)$ ceilingKey(K key) – Lock-free search for least >= key – $O(\log n)$ lowerKey(K key) – Lock-free search for greatest strictly < key – $O(\log n)$ higherKey(K key) – Lock-free search for least strictly > key – $O(\log n)$ putIfAbsent(K,V) – CAS-based conditional insert – $O(\log n)$ replace(K,V) – CAS-based conditional update – $O(\log n)$ size() – Traversal count (not $O(1)$) – $O(n)$ descendingMap() – View with reversed key ordering – $O(1)$ keySet() – Weakly consistent ascending key set – $O(1)$

Class / Interface	Key Methods Short Description Time Complexity
EnumMap<K,V>	get(Object key) – Array lookup by key.ordinal() – O(1) put(K key, V value) – Stores in values[key.ordinal()] – O(1) remove(Object key) – Nullifies values[key.ordinal()]; updates size – O(1) containsKey(Object key) – Checks values[key.ordinal()] != null – O(1) containsValue(Object val) – Linear scan of values[] – O(n) size() – Returns count of non-null slots – O(1) isEmpty() – Returns size == 0 – O(1) clear() – Nullifies all values[]; resets size – O(n) keySet() – Returns keys in enum-declaration order – O(1) values() – Returns values in enum-declaration order – O(1) entrySet() – Returns entries in enum-declaration order – O(1) putAll(Map m) – Copies all entries (must be EnumMap or same type) – O(n) clone() – Shallow copy of this EnumMap – O(n)
WeakHashMap<K,V>	get(Object key) – Purges stale refs first; hash lookup – O(1) avg put(K key, V value) – Purges stale refs; wraps key in WeakReference; inserts – O(1) avg remove(Object key) – Purges stale refs; removes entry – O(1) avg containsKey(Object key) – Purges stale refs; hash lookup – O(1) avg containsValue(Object val) – Full scan; purges stale refs – O(n) size() – Purges stale refs; returns count – O(n) [expunge] isEmpty() – Purges stale refs; checks size == 0 – O(n) [expunge] clear() – Drops all bucket arrays; clears reference queue – O(n) keySet() – Set view; keys may disappear under GC pressure – O(1) values() – Collection view; entries may disappear under GC – O(1) entrySet() – Set<Entry> view; stale entries auto-expunged – O(1) putAll(Map m) – Copies all entries wrapping keys as WeakReferences – O(k)
Hashtable<K,V>	get(Object key) – synchronized hash lookup – O(1) avg sync put(K key, V value) – synchronized insert/update; no null key/value – O(1) avg sync remove(Object key) – synchronized removal – O(1) avg sync containsKey(Object key) – synchronized hash lookup – O(1) avg sync containsValue(Object val) – synchronized linear scan – O(n) sync contains(Object val) – Legacy; same as containsValue (synchronized) – O(n) sync size() – synchronized count – O(1) sync isEmpty() – synchronized size == 0 check – O(1) sync clear() – synchronized; nullifies all buckets – O(capacity) sync keys() – Returns legacy Enumeration of keys – O(1) elements() – Returns legacy Enumeration of values – O(1) keySet() – Returns synchronized Set view of keys – O(1) entrySet() – Returns synchronized Set<Entry> view – O(1)
Properties	getProperty(String key) – Looks up key; falls back to defaults chain – O(1) avg sync getProperty(String key, String d) – Returns value or default if absent – O(1) avg sync setProperty(String key, String v) – Calls put(); no null key/value – O(1) avg sync load(InputStream in) – Parses key=value pairs from byte stream – O(n) load(Reader reader) – Parses key=value pairs from char stream – O(n) store(OutputStream out, String c) – Writes key=value pairs to byte stream – O(n) store(Writer writer, String c) – Writes key=value pairs to char stream – O(n) loadFromXML(InputStream in) – Parses XML properties document – O(n) storeToXML(OutputStream,String) – Writes properties as XML document – O(n) stringPropertyNames() – Returns Set of String keys (incl. defaults) – O(n) list(PrintStream out) – Prints all properties (debug utility) – O(n) defaults field – Parent Properties for fallback chain – O(1)

Class / Interface	Key Methods Short Description Time Complexity
ConcurrentMap<K,V>	putIfAbsent(K key, V val) – Atomically inserts only if key absent; returns old val – O(varies) remove(K key, V val) – Atomically removes only if key maps to exact value – O(varies) replace(K key, V val) – Atomically replaces only if key is present – O(varies) replace(K,V old, V new) – Atomically replaces only if mapped to old value – O(varies) get(Object key) – Inherited; thread-safe read – O(varies) put(K key, V value) – Inherited; thread-safe write – O(varies) remove(Object key) – Inherited; thread-safe removal – O(varies) containsKey(Object key) – Inherited; thread-safe lookup – O(varies) size() – Inherited; approximate count – O(varies) forEach(BiConsumer) – Thread-safe iteration (weakly consistent) – O(n) compute(K,BiFunction) – Atomically compute new value for key – O(varies) merge(K,V,BiFunction) – Atomically merge value for key – O(varies)
ConcurrentHashMap<K,V>	get(Object key) – Lock-free CAS read from bin/tree node – O(1) avg put(K key, V value) – CAS on empty bin; synchronized on head node if collision – O(1) avg remove(Object key) – Synchronized on bin head; CAS-based unlink – O(1) avg putIfAbsent(K,V) – CAS-based conditional insert – O(1) avg replace(K,V) – Synchronized conditional update – O(1) avg replace(K,V old,V new) – Synchronized CAS value swap – O(1) avg containsKey(Object key) – Lock-free hash + bin scan – O(1) avg size() / mappingCount() – Sums CounterCell[] array (not exact under concurrency) – O(segments) newKeySet() – Returns ConcurrentHashMap-backed Set view – O(1) keySet() – Returns view of keys (weakly consistent) – O(1) entrySet() – Returns entry set view (weakly consistent) – O(1) forEach(long, BiConsumer) – Parallel forEach with parallelism threshold – O(n) search(long, BiFunction) – Parallel search returning first non-null result – O(n) reduce(long,BiFun,BiFunc) – Parallel map-reduce over entries – O(n) compute(K,BiFunction) – Synchronized compute for key – O(1) avg merge(K,V,BiFunction) – Synchronized merge for key – O(1) avg

Diagram 2 – Underlying Data Structures

Java Backend Architecture Interview Series - Chapter 1: Java Collections Deep Dive

Interface	Abstract Class	Concrete Class	Concurrent/Thread-Safe	Deprecated/Legacy
Class / Interface (indent = hierarchy depth)		Underlying Data Structure		
Iterable<T>		Backed by nothing (contract only)		
Collection<E>		Backed by nothing (contract only)		
List<E>		Backed by nothing (contract only)		
ArrayList<E>		Dynamic Object[] array		
LinkedList<E>		Doubly-linked node list		
Vector<E>		Dynamic Object[] array (like ArrayList)		
Stack<E>		Inherits Vector's Object[]		
CopyOnWriteArrayList<E>		Immutable snapshot Object[] per write		
Set<E>		Backed by nothing (contract only)		
HashSet<E>		Backed by HashMap<E, Object>		
LinkedHashSet<E>		Backed by LinkedHashMap (doubly-linked buckets)		
SortedSet<E>		Backed by nothing (contract only)		
NavigableSet<E>		Backed by nothing (contract only)		
TreeSet<E>		Backed by TreeMap (Red-Black tree)		
ConcurrentSkipListSet<E>		Backed by ConcurrentSkipListMap (skip list)		
EnumSet<E>		Bit-vector (long[] or single long)		
CopyOnWriteArraySet<E>		Backed by CopyOnWriteArrayList		
Queue<E>		Backed by nothing (contract only)		
PriorityQueue<E>		Min-heap (Object[] binary heap)		
Deque<E>		Backed by nothing (contract only)		
ArrayDeque<E>		Circular resizable Object[]		
BlockingQueue<E>		Backed by nothing (contract only)		
ArrayBlockingQueue<E>		Bounded circular Object[]		
LinkedBlockingQueue<E>		Linked node list (optionally bounded)		
PriorityBlockingQueue<E>		Unbounded min-heap (Object[])		
SynchronousQueue<E>		No internal storage (direct transfer)		
DelayQueue<E>		PriorityQueue of Delayed elements		
LinkedTransferQueue<E>		Unbounded linked nodes (FIFO)		
Map<K, V>		Backed by nothing (contract only)		
HashMap<K, V>		Array of linked-list buckets; treeified at 8+ entries		
LinkedHashMap<K, V>		HashMap + doubly-linked entry list		
SortedMap<K, V>		Backed by nothing (contract only)		
NavigableMap<K, V>		Backed by nothing (contract only)		

Class / Interface (indent = hierarchy depth)	Underlying Data Structure
TreeMap<K,V>	Red-Black tree of Entry nodes
ConcurrentSkipListMap<K,V>	Skip list (tower of linked nodes)
EnumMap<K,V>	Object[] indexed by enum ordinal
WeakHashMap<K,V>	Array of WeakReference buckets
Hashtable<K,V>	Array of linked-list buckets (no treeify)
Properties	Inherits Hashtable's array of buckets
ConcurrentMap<K,V>	Backed by nothing (contract only)
ConcurrentHashMap<K,V>	Array of Node bins; treeified at 8+ (CAS lock-free reads)

Diagram 3 – Thread Safety & Concurrency Mapping

Java Backend Architecture Interview Series - Chapter 1: Java Collections Deep Dive

Interface	Abstract Class	Concrete Class	Concurrent/Thread-Safe	Deprecated/Legacy
Class / Interface (indent = hierarchy depth)		Thread Safety / Concurrency		
Iterable<T>		Not thread-safe (contract only)		
Collection<E>		Not thread-safe (contract only)		
List<E>		Not thread-safe (contract only)		
ArrayList<E>		Not thread-safe		
LinkedList<E>		Not thread-safe		
Vector<E>		Thread-safe (synchronized methods)		
Stack<E>		Thread-safe (via Vector)		
CopyOnWriteArrayList<E>		Thread-safe (lock on write, lock-free read)		
Set<E>		Not thread-safe (contract only)		
HashSet<E>		Not thread-safe		
LinkedHashSet<E>		Not thread-safe		
SortedSet<E>		Not thread-safe (contract only)		
NavigableSet<E>		Not thread-safe (contract only)		
TreeSet<E>		Not thread-safe		
ConcurrentSkipListSet<E>		Thread-safe (lock-free CAS)		
EnumSet<E>		Not thread-safe		
CopyOnWriteArraySet<E>		Thread-safe (copy-on-write)		
Queue<E>		Not thread-safe (contract only)		
PriorityQueue<E>		Not thread-safe		
Deque<E>		Not thread-safe (contract only)		
ArrayDeque<E>		Not thread-safe		
BlockingQueue<E>		Thread-safe (contract)		
ArrayBlockingQueue<E>		Thread-safe (single ReentrantLock)		
LinkedBlockingQueue<E>		Thread-safe (separate put/take locks)		
PriorityBlockingQueue<E>		Thread-safe (single ReentrantLock)		
SynchronousQueue<E>		Thread-safe (CAS-based)		
DelayQueue<E>		Thread-safe (ReentrantLock)		
LinkedTransferQueue<E>		Thread-safe (CAS lock-free)		
Map<K,V>		Not thread-safe (contract only)		
HashMap<K,V>		Not thread-safe		
LinkedHashMap<K,V>		Not thread-safe		
SortedMap<K,V>		Not thread-safe (contract only)		
NavigableMap<K,V>		Not thread-safe (contract only)		

Class / Interface (indent = hierarchy depth)	Thread Safety / Concurrency
TreeMap<K,V>	Not thread-safe
ConcurrentSkipListMap<K,V>	Thread-safe (lock-free CAS)
EnumMap<K,V>	Not thread-safe
WeakHashMap<K,V>	Not thread-safe
Hashtable<K,V>	Thread-safe (synchronized methods)
Properties	Thread-safe (via Hashtable)
ConcurrentMap<K,V>	Thread-safe (contract)
ConcurrentHashMap<K,V>	Thread-safe (CAS + bin-level sync)

Diagram 4 – Memory Footprint & Spatial Complexity

Java Backend Architecture Interview Series - Chapter 1: Java Collections Deep Dive

Interface	Abstract Class	Concrete Class	Concurrent/Thread-Safe	Deprecated/Legacy
Class / Interface (indent = hierarchy depth)		Memory / Spatial Complexity		
Iterable<T>		Zero fields; pure interface		
Collection<E>		Zero fields; pure interface		
List<E>		Zero fields; pure interface		
ArrayList<E>		O(n) – array + ~50% slack capacity		
LinkedList<E>		O(n) – each node ~40 bytes (header+prev+next+data)		
Vector<E>		O(n) – same as ArrayList; doubles on resize		
Stack<E>		O(n) – same as Vector		
CopyOnWriteArrayList<E>		O(n) – two arrays exist briefly during copy		
Set<E>		Zero fields; pure interface		
HashSet<E>		O(n) – HashMap overhead ~48B/entry		
LinkedHashSet<E>		O(n) – extra prev/next pointers vs HashSet		
SortedSet<E>		Zero fields; pure interface		
NavigableSet<E>		Zero fields; pure interface		
TreeSet<E>		O(n) – each node ~48 bytes		
ConcurrentSkipListSet<E>		O(n log n) – skip list levels add pointer overhead		
EnumSet<E>		O(1) – 64 enums fit in one long; minimal memory		
CopyOnWriteArraySet<E>		O(n) – two arrays during copy		
Queue<E>		Zero fields; pure interface		
PriorityQueue<E>		O(n) – Object[] with heap layout		
Deque<E>		Zero fields; pure interface		
ArrayDeque<E>		O(n) – circular array; no wasted nodes		
BlockingQueue<E>		Zero fields; pure interface		
ArrayBlockingQueue<E>		O(capacity) – fixed array; predictable memory		
LinkedBlockingQueue<E>		O(n) – nodes ~32 bytes each		
PriorityBlockingQueue<E>		O(n) – heap grows unbounded		
SynchronousQueue<E>		O(1) – stores zero elements		
DelayQueue<E>		O(n) – PriorityQueue inside		
LinkedTransferQueue<E>		O(n) – linked nodes		
Map<K,V>		Zero fields; pure interface		
HashMap<K,V>		O(n) – 16×initial capacity bytes + 48B/entry		
LinkedHashMap<K,V>		O(n) – extra prev/next per entry vs HashMap		
SortedMap<K,V>		Zero fields; pure interface		
NavigableMap<K,V>		Zero fields; pure interface		

Class / Interface (indent = hierarchy depth)	Memory / Spatial Complexity
TreeMap<K,V>	$O(n)$ – each Entry ~48 bytes
ConcurrentSkipListMap<K,V>	$O(n \log n)$ – skip list pointers add overhead
EnumMap<K,V>	$O(EnumType)$ – fixed array; very compact
WeakHashMap<K,V>	$O(n)$ – weakly reachable keys reclaimed by GC
Hashtable<K,V>	$O(n)$ – similar to HashMap
Properties	$O(n)$ – same as Hashtable
ConcurrentMap<K,V>	Zero fields; pure interface
ConcurrentHashMap<K,V>	$O(n)$ – similar to HashMap; ~12B overhead per node

Diagram 5 – Null Acceptance & Ordering Guarantees

Java Backend Architecture Interview Series - Chapter 1: Java Collections Deep Dive

Interface	Abstract Class	Concrete Class	Concurrent/Thread-Safe	Deprecated/Legacy
Class / Interface (indent = hierarchy depth)		Null Policy & Ordering		
Iterable<T>		No null constraint defined		
Collection<E>		Null policy left to implementors		
List<E>		Allows nulls & duplicates; insertion-ordered		
ArrayList<E>		Allows nulls & duplicates; insertion-ordered		
LinkedList<E>		Allows nulls & duplicates; insertion-ordered		
Vector<E>		Allows nulls & duplicates; insertion-ordered		
Stack<E>		Allows nulls; LIFO ordering		
CopyOnWriteArrayList<E>		Allows nulls & duplicates; insertion-ordered		
Set<E>		No duplicates; null policy per impl		
HashSet<E>		Allows one null; no ordering guarantee		
LinkedHashSet<E>		Allows one null; insertion-ordered		
SortedSet<E>		No nulls (Comparator must not allow null); sorted		
NavigableSet<E>		No nulls; sorted (ascending/descending views)		
TreeSet<E>		No nulls (unless Comparator allows); sorted order		
ConcurrentSkipListSet<E>		No nulls; sorted ascending		
EnumSet<E>		No nulls; enum-declaration ordered		
CopyOnWriteArraySet<E>		Allows one null; insertion-ordered (no duplicates)		
Queue<E>		Null policy per impl; FIFO typical		
PriorityQueue<E>		No nulls; priority (natural/Comparator) ordered		
Deque<E>		Null policy per impl; both-end access		
ArrayDeque<E>		No nulls; insertion order (FIFO or LIFO per use)		
BlockingQueue<E>		No nulls (spec); FIFO typical		
ArrayBlockingQueue<E>		No nulls; FIFO ordered		
LinkedBlockingQueue<E>		No nulls; FIFO ordered		
PriorityBlockingQueue<E>		No nulls; priority ordered (not FIFO)		
SynchronousQueue<E>		No nulls; no ordering (size always 0)		
DelayQueue<E>		No nulls; delay-expiry ordered		
LinkedTransferQueue<E>		No nulls; FIFO ordered		
Map<K,V>		Null policy per impl; no ordering guarantee		
HashMap<K,V>		Allows one null key & null values; no order		
LinkedHashMap<K,V>		Allows one null key & null values; insertion or access order		
SortedMap<K,V>		No null keys (Comparator); sorted by key		
NavigableMap<K,V>		No null keys; sorted ascending/descending views		

Class / Interface (indent = hierarchy depth)	Null Policy & Ordering
TreeMap<K,V>	No null keys; sorted by natural/Comparator order
ConcurrentSkipListMap<K,V>	No null keys or values; sorted ascending
EnumMap<K,V>	No null keys; null values allowed; declaration ordered
WeakHashMap<K,V>	Allows null key & null values; no ordering; entries may vanish
Hashtable<K,V>	No null keys or values; no ordering
Properties	No null keys or values; no ordering
ConcurrentMap<K,V>	No null keys or values (contract); no ordering
ConcurrentHashMap<K,V>	No null keys or values; no ordering guarantee

Diagram 6 – Iterator & Iterable Roots

Java Backend Architecture Interview Series - Chapter 1: Java Collections Deep Dive

Interface	Abstract Class	Concrete Class	Concurrent/Thread-Safe	Deprecated/Legacy
Class / Interface (indent = hierarchy depth)		Iterator / Iterable Integration		
Iterable<T>		Root of for-each; defines iterator()		
Collection<E>		Extends Iterable; adds bulk ops		
List<E>		Extends Collection; adds positional access		
ArrayList<E>		Uses Itr / ListItr inner classes		
LinkedList<E>		Uses ListItr inner class; also implements Deque		
Vector<E>		Enumeration (legacy) + ListItr		
Stack<E>		Inherits Vector iterators		
CopyOnWriteArrayList<E>		Snapshot iterator; never ConcurrentModificationException		
Set<E>		Extends Collection; no new iterator methods		
HashSet<E>		Uses HashMap.KeyIterator		
LinkedHashSet<E>		Uses LinkedHashMap.LinkedKeyIterator		
SortedSet<E>		Extends Set; adds range-view methods		
NavigableSet<E>		Extends SortedSet; adds navigation methods		
TreeSet<E>		Uses TreeMap.KeyIterator (in-order traversal)		
ConcurrentSkipListSet<E>		Uses ConcurrentSkipListMap.KeyIterator (weakly consistent)		
EnumSet<E>		Uses RegularEnumSet.EnumSetIterator		
CopyOnWriteArraySet<E>		Delegates to CopyOnWriteArrayList snapshot iterator		
Queue<E>		Extends Collection; adds offer/poll/peek		
PriorityQueue<E>		Uses Itr / ArrayIterator inner classes		
Deque<E>		Extends Queue; adds head/tail ops		
ArrayDeque<E>		Uses DeqIterator / DescendingIterator		
BlockingQueue<E>		Extends Queue; adds blocking put/take		
ArrayBlockingQueue<E>		Uses Itr inner class; weakly consistent		
LinkedBlockingQueue<E>		Uses Itr inner class; weakly consistent		
PriorityBlockingQueue<E>		Uses Itr; weakly consistent		
SynchronousQueue<E>		Uses EmptyIterator (always empty)		
DelayQueue<E>		Delegates to PriorityQueue iterator		
LinkedTransferQueue<E>		Uses Itr; weakly consistent		
Map<K,V>		keySet()/values()/entrySet() return Collection views		
HashMap<K,V>		Uses KeyIterator / ValueIterator / EntryIterator		
LinkedHashMap<K,V>		Uses LinkedHashMapIterator subclasses		
SortedMap<K,V>		Extends Map; adds range-view (headMap/tailMap)		
NavigableMap<K,V>		Extends SortedMap; adds navigation ops		

Class / Interface (indent = hierarchy depth)	Iterator / Iterable Integration
TreeMap<K,V>	Uses EntryIterator (in-order)
ConcurrentSkipListMap<K,V>	Uses KeyIterator / EntryIterator (weakly consistent)
EnumMap<K,V>	Uses EntryIterator over values array
WeakHashMap<K,V>	Uses EntryIterator; expunges stale entries on access
Hashtable<K,V>	Enumerator (legacy) + EntryIterator
Properties	Inherits Hashtable iterators
ConcurrentMap<K,V>	Extends Map; adds atomic conditional ops
ConcurrentHashMap<K,V>	Uses EntryIterator / KeyIterator (weakly consistent)

Diagram 7 – Core Architectural Utility & Role

Java Backend Architecture Interview Series - Chapter 1: Java Collections Deep Dive

Interface	Abstract Class	Concrete Class	Concurrent/Thread-Safe	Deprecated/Legacy
Class / Interface (indent = hierarchy depth)		Architectural Role & Usage		
Iterable<T>		Enables enhanced for-loop on any object		
Collection<E>		Core collection contract: add/remove/contains/size		
List<E>		Ordered, index-based collection contract		
ArrayList<E>		Default go-to List; best for random access		
LinkedList<E>		Dual-purpose List+Deque; great for queue/stack use		
Vector<E>		Legacy synchronized List; prefer CopyOnWriteArrayList		
Stack<E>		Legacy LIFO stack; prefer ArrayDeque		
CopyOnWriteArrayList<E>		Read-heavy concurrent List; safe iteration without locks		
Set<E>		No-duplicate collection contract		
HashSet<E>		General-purpose O(1) Set; backed by hash table		
LinkedHashSet<E>		Insertion-ordered Set with O(1) ops		
SortedSet<E>		Sorted set contract with natural/custom ordering		
NavigableSet<E>		Navigation-capable sorted set (floor/ceiling etc.)		
TreeSet<E>		Sorted NavigableSet backed by Red-Black tree		
ConcurrentSkipListSet<E>		Concurrent NavigableSet; lock-free sorted ops		
EnumSet<E>		Ultra-compact Set for enum types; bit-mask backed		
CopyOnWriteArraySet<E>		Thread-safe Set for tiny, read-heavy collections		
Queue<E>		FIFO queue contract		
PriorityQueue<E>		Unbounded priority queue (min-heap); not FIFO		
Deque<E>		Double-ended queue contract (stack + queue)		
ArrayDeque<E>		Preferred stack/queue; faster than Stack & LinkedList		
BlockingQueue<E>		Producer-consumer blocking queue contract		
ArrayBlockingQueue<E>		Bounded blocking FIFO queue for thread handoff		
LinkedBlockingQueue<E>		High-throughput bounded/unbounded blocking queue		
PriorityBlockingQueue<E>		Unbounded priority-ordered blocking queue		
SynchronousQueue<E>		Zero-capacity rendezvous channel for thread handoff		
DelayQueue<E>		Schedule future tasks; elements released after delay		
LinkedTransferQueue<E>		High-perf transfer queue; direct producer-consumer handoff		
Map<K,V>		Key-value mapping contract; foundation of all Maps		
HashMap<K,V>		General-purpose hash map; most used Map implementation		
LinkedHashMap<K,V>		Insertion/access-ordered Map; basis for LRU caches		
SortedMap<K,V>		Sorted map contract with natural/custom key ordering		
NavigableMap<K,V>		Navigation-capable sorted map (floor/ceiling etc.)		

Class / Interface (indent = hierarchy depth)	Architectural Role & Usage
TreeMap<K,V>	Sorted NavigableMap backed by Red-Black tree
ConcurrentSkipListMap<K,V>	Concurrent NavigableMap; lock-free sorted ops
EnumMap<K,V>	Most efficient Map for enum keys; ordinal-indexed
WeakHashMap<K,V>	Keys held weakly; auto-removed when GC'd (caches)
Hashtable<K,V>	Legacy synchronized Map; prefer ConcurrentHashMap
Properties	String key-value store for config files (.properties)
ConcurrentMap<K,V>	Atomic conditional ops contract for concurrent maps
ConcurrentHashMap<K,V>	High-concurrency Map; CAS reads, bin-level writes